

CLAP: Direct VLM-to-VLA Adaptation via Language-Action Grounding

Anonymous Author(s)

Affiliation

Address

email

Abstract: Vision-language-action models (VLAs) inherit semantic capabilities from pretrained VLMs, yet large-scale post-training on robot data and architectural modifications can reshape the backbone so extensively that it becomes difficult to isolate what the VLM contributes to control. Directly converting pretrained VLMs into VLAs with minimal architectural change offers a more transparent path to understanding how VLM capabilities transfer across model scales. The core obstacle is output-distribution mismatch: predicting actions as bare numeric token sequences moves generation away from the VLM’s pretrained language distribution, degrading the capabilities we seek to preserve. To address this, we propose **CLAP** (Causal Language-Action Prediction), which prepends each numeric action sequence with a natural-language action description, causally conditioning precise action-token prediction on a language-action plan without modifying the backbone architecture. With single-epoch fine-tuning alone, 2B CLAP achieves 90.8% on LIBERO (+14.9 pt over VLA-0) and improves robustness on LIBERO-PRO under language, object, and spatial perturbations. We release CLAP at 0.8B, 2B, and 4B as an open-weight, multi-scale compact VLA family from a single VLM lineage, enabling controlled analysis of VLM-to-VLA capability transfer.

Keywords: Vision-Language-Action Models, Robot Manipulation

1 Introduction

Rapid progress in vision-language models (VLMs) continually expands the capabilities of robot *perception* and *reasoning*, yet transferring these capabilities to robot *control* remains disproportionately difficult. Vision-language-action models (VLAs), which sit at the intersection of vision-language modeling and robot learning, pursue this transfer by fine-tuning pretrained VLMs on robot demonstrations [1, 2, 3]. Recent systems have improved action prediction through discretized action tokens, learned action experts, or diffusion action heads [1, 4], but each addition moves the VLA further from its underlying VLM in both architecture and training distribution. This architectural and data complexity makes it difficult to isolate what the pretrained VLM actually contributes to control, limiting both systematic capability analysis and the transfer of advances from the broader VLM community. *What would it take to make VLA research as accessible as VLM research?*

A central source of complexity in VLM-to-VLA adaptation is an *output-distribution mismatch*: standard VLA fine-tuning trains a VLM pretrained to generate semantically structured language to instead emit bare action-token sequences such as 4 12 98 3 0 0. These tokens carry little of the linguistic or spatial structure seen during pretraining, so learning to act can degrade the representations that support semantic generalization. This unresolved mismatch is especially important for *compact* VLAs, which use lightweight VLM backbones for efficient deployment but may be more sensitive to distribution shift during fine-tuning [5, 6]. Recent architecture-free VLM-to-VLA work provides cleaner settings for studying this issue by avoiding separate action experts, but existing approaches either predict bare action tokens [7] or replace numeric actions with language-only de-

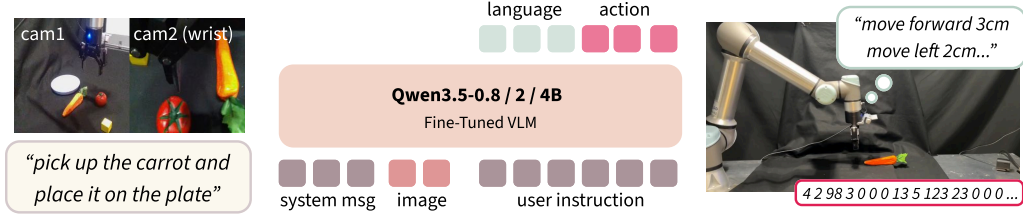


Figure 1: CLAP converts a pretrained VLM directly into a deployable VLA by prepending a natural-language action description to numeric action tokens, keeping every prediction step closer to the VLM pretraining distribution. CLAP requires no action expert or architectural change.

39 scriptions [8]. These choices leave a trade-off between direct executability and alignment with the
40 VLM’s pretrained language distribution.

41 To address this mismatch, we propose **CLAP** (Causal Language-Action Prediction), a lightweight
42 VLM-to-VLA fine-tuning recipe that **reformulates action prediction as language-conditioned ac-**
43 **tion generation**. Instead of predicting bare action tokens directly or replacing actions with language-
44 only descriptions, CLAP generates both in a single autoregressive sequence: before predicting dis-
45 crete numeric action tokens, the model first generates a natural-language action description, such as
46 “*move forward, tilt right, and close the gripper*”. Because the full sequence is generated autoregres-
47 sively, causal attention conditions the numeric action tokens on the preceding language description,
48 making it a conditioning intermediate rather than an auxiliary prediction. CLAP requires **no action**
49 **expert, vocabulary modification, architectural change, or manual annotation**, while retaining
50 executable numeric action tokens for precise control.

51 We evaluate CLAP across manipulation, robustness, and capability-transfer analysis using compact
52 Qwen3.5 backbones at 0.8B, 2B, and 4B parameters [9]. On LIBERO, 2B CLAP achieves 90.8%
53 success with a single fine-tuning epoch, outperforming VLA-0 by +14.9 percentage points under
54 identical conditions and approaching VLA-0’s 64-epoch performance, showing that the language-
55 action prefix substantially improves learning efficiency. On LIBERO-PRO, CLAP improves robust-
56 ness under instruction, object, and spatial perturbations, suggesting that the prefix helps beyond
57 in-distribution imitation. On VLABench, the 2B backbone performs strongest across most capa-
58 bility dimensions and also yields the strongest CLAP policy, indicating that parameter count alone
59 does not determine transfer quality in the compact regime. Together, these evaluations suggest that
60 CLAP provides not only an efficient compact VLA recipe, but also a controlled setting for studying
61 how VLM capabilities transfer to robot control.

62 In summary, our main contribution is **CLAP**: a minimal VLM-to-VLA recipe that changes only
63 the output representation, jointly generating language-action descriptions and action tokens without
64 an action expert, vocabulary modification, or architectural change. Consistent with this lightweight
65 design, CLAP can be trained across 0.8B, 2B, and 4B model sizes within 6.5 hours on $8 \times \text{H200}$
66 GPUs. We will release weights, code, and training recipes to support future research on VLM-to-
67 VLA adaptation.

68 2 Related Work

69 **Vision-language-action models and action representations.** Vision-language-action (VLA)
70 models adapt pretrained vision-language models (VLMs) [10, 11, 12] to robot control by com-
71 bining internet-scale vision-language pretraining with behavioral cloning on robot demonstra-
72 tions [13, 14, 15]. A central design choice is how to represent low-level actions within or alongside
73 the VLM. One strategy discretizes continuous actions into tokens, casting robot control as autore-
74 gressive next-token prediction [4, 7, 16]. Another adds dedicated diffusion or flow-matching action
75 heads to generate continuous actions directly [1, 2, 17]. CLAP builds on the autoregressive direc-

tion, but changes the output representation itself: instead of predicting bare discrete action tokens, it inserts a language-action prefix before the action tokens without adding an action expert.

Language grounding and intermediate reasoning. Prior work has used language, affordances, and intermediate reasoning traces to make robot policies more semantically structured. One line of work conditions manipulation policies or high-level plans on natural language, often grounding instructions through learned affordances or embodied multimodal representations [18, 19, 20, 21]. More recent VLA methods move language or trace-like intermediates closer to action prediction. VLM2VLA [8] represents actions directly as natural-language text, improving alignment with the VLM pretraining distribution but limiting low-level control precision. Embodied reasoning methods such as ECoT [22], ECoT-Lite [23], and TraceVLA [24] rely on auxiliary intermediate representations, including reasoning tokens and visual traces, before action generation, but often incur additional supervision, prompting, or inference-time overhead. LAP [3] uses language-action descriptions as a pretraining signal, but still relies on an action expert for final action generation. By contrast, CLAP places the language-action description directly in the final autoregressive policy output, where it conditions subsequent discrete action tokens while preserving executable numeric control.

Capability preservation. Fine-tuning VLMs on robot action data can disrupt pretrained representations, degrading capabilities such as multimodal understanding, spatial reasoning, and generalization [25, 26]. One line of work addresses this by insulating or preserving the VLM during policy learning. Knowledge Insulation [25] blocks gradients from the action expert, FLOWER [27] decouples action generation from the VLM via an intermediate-fusion Flow Transformer, and GenVLA [26] uses a dual-encoder design with vision-language co-training. These methods improve preservation, but introduce additional modules, training objectives, gradient-blocking schemes, or co-training data. CLAP instead asks whether capability-preserving VLM-to-VLA adaptation can be improved by changing only the output representation while keeping the VLM architecture and training objective unchanged.

Compact VLAs. Capability preservation is especially important for compact VLAs, where efficient deployment must be balanced against limited model capacity. SmolVLA [5], TinyVLA [6], and VLA-Adapter [28] improve efficiency through lightweight backbones, adapter-based tuning, or specialized policy modules. However, they either focus on a specific model scale or rely on additional policy/action components. In contrast, CLAP applies the same architecture-preserving recipe across 0.8B, 2B, and 4B Qwen3.5 backbones, enabling controlled analysis of VLM-to-VLA transfer across compact scales.

3 Preliminaries

Given an image observation o and a natural-language instruction l , an autoregressive VLA models a robot policy π_θ by predicting an action sequence. Each h -step action chunk $\mathbf{a} = (a_1, \dots, a_h)$ consists of h sequential 7-DoF commands, where each $a_t \in \mathbb{R}^7$ encodes end-effector translation, rotation, and gripper state. Following prior work [7], each action dimension is discretized into $K = 1000$ integer bins and represented as text tokens using the VLM’s existing vocabulary. The policy predicts the flattened action-token sequence autoregressively:

$$\pi_\theta(\mathbf{a} \mid o, l) = \prod_{i=1}^{7h} \pi_\theta(a^{(i)} \mid a^{(<i)}, o, l), \quad (1)$$

where $a^{(i)}$ denotes the i -th token in the flattened action sequence and $a^{(<i)}$ denotes all preceding action tokens. The VLM backbone is thus fine-tuned with the same next-token prediction objective as language pretraining, while its outputs can be directly decoded into robot commands. However, the fine-tuning targets are bare integer sequences that lack the semantic structure of natural language, diverging from the distribution the backbone was pretrained to generate. We refer to this shift as

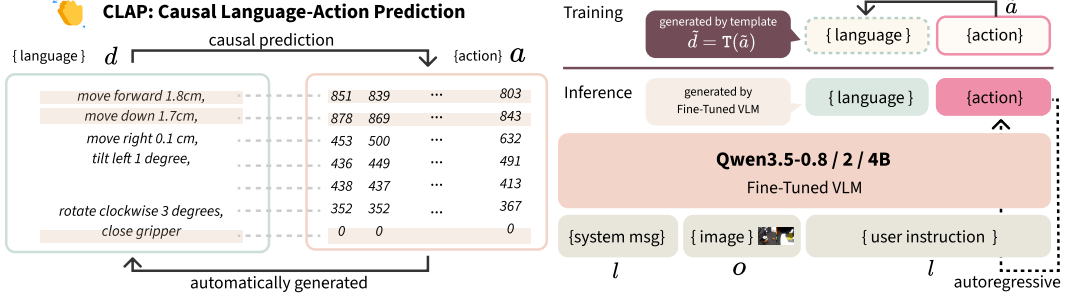


Figure 2: **Overview of CLAP.** CLAP changes only the target output sequence for VLA fine-tuning. During training, the language-action description $\tilde{d}$ is generated from the ground-truth action chunk by a fixed template $\tilde{d} = T(\tilde{a})$. At inference time, both language-action description d and the numeric action tokens a are generated autoregressively by the model. Because generation is causal, each action token conditions on the preceding language-action description.

121 *output-distribution mismatch* [3, 8]. CLAP mitigates this mismatch by incorporating a language-
 122 grounded action description into the autoregressive output.

123 4 CLAP: Causal Language-Action Prediction

124 We use the output-distribution mismatch introduced in Section 3 to design a simple training recipe
 125 for compact VLA policies. Our goal is to retain the semantic generalization of the pretrained VLM
 126 while producing directly executable low-level robot actions, without requiring an action expert,
 127 vocabulary extension, or large-scale robot pretraining. Figure 2 provides an overview.

128 **Language-action representation.** The central design question in converting a pretrained VLM
 129 into a VLA is what *action representation* to use. Representing actions as bare numeric tokens
 130 yields compact, directly executable outputs, but moves generation away from the VLM’s pretrained
 131 language distribution. A promising alternative, introduced by LAP [3], is to express actions as
 132 natural-language descriptions, preserving closer alignment with the VLM’s pretrained generation
 133 behavior. However, language-only action descriptions are too verbose to specify precise low-level
 134 commands efficiently, and still require an external action generator to produce executable robot
 135 actions [3, 25]. CLAP addresses this trade-off by prepending a short language-action description to
 136 the numeric action tokens within a single autoregressive sequence, extending Eq. 1 as

$$\pi_{\theta}(d, \mathbf{a} \mid o, l) = \underbrace{\prod_{j=1}^{|d|} \pi_{\theta}(d^{(j)} \mid d^{(<j)}, o, l)}_{\text{language-action prefix}} \cdot \underbrace{\prod_{i=1}^{7h} \pi_{\theta}(a^{(i)} \mid a^{(<i)}, d, o, l)}_{\text{numeric action tokens}}, \quad (2)$$

137 where d is a natural-language description of the action chunk $\mathbf{a}$. During training, the description
 138 is generated deterministically from the ground-truth action chunk $\tilde{\mathbf{a}}$ by a fixed template as $\tilde{d} =$
 139 $T(\tilde{\mathbf{a}})$. At inference time, the model generates both d and $\mathbf{a}$ autoregressively. This representation
 140 satisfies three desiderata: (i) semantic alignment with the VLM’s pretrained distribution, (ii) direct
 141 executability without an external action generator, and (iii) token efficiency for real-time control.

142 **Template-based language-action prefix.** We instantiate the language component of this repre-
 143 sentation with a deterministic template. For each h -step action chunk, CLAP prepends a compact
 144 natural-language summary of the intended motion, such as "move forward 3 cm, tilt right
 145 1 degree, close gripper". This prefix summarizes the chunk-level translation, rotation, and
 146 gripper command, while the following numeric tokens encode precise low-level action values. Be-
 147 cause the prefix is generated from the ground-truth action chunk using a fixed template, CLAP
 148 requires no manual annotation beyond the original robot demonstrations. Table 1 shows an example
 149 output sequence generated from an input image observation and language instruction.

System Message:

Analyze the input image and predict robot actions for the next 8 timesteps. Each action has 7 dimensions. First describe the motion in natural language inside `<think> ... </think>`. Then output exactly 56 integers (0-1000 each), space-separated, for the discretized actions in timestep order. Nothing else.

User Instruction: grasp cube and drop it onto the plate.

Language-Action Prediction:

`<think>` move back 0.5 cm, move up 3.5 cm, move left 0.4 cm, tilt left 1 degrees, tilt back 1 degrees, rotate counterclockwise 3 degrees, open gripper
`</think>`
408 980 166 319 505 491 1000 407 980 159 312 496 486 1000 406 980 152 303 486 479
1000 406 981 150 300 489 480 1000 407 980 149 294 484 483 1000 408 980 146 289
482 480 1000 410 979 144 287 486 483 1000 412 978 137 277 484 479 1000

Table 1: **Example CLAP output sequence.** Given an instruction, system message, and image observations, CLAP first generates a fixed-template language-action prefix that summarizes the intended h -step motion, then generates discrete numeric action tokens for execution. The prefix provides a structured intermediate conditioned on the observation and instruction, while the numeric tokens encode the precise 7-DoF command for each step.

Joint generation and training. Given the template-derived prefix, CLAP trains the model to predict a single concatenated sequence: language-action description tokens followed by numeric action tokens. We optimize the standard autoregressive cross-entropy objective

$$\mathcal{L}_{\text{CLAP}} = -\mathbb{E}_{(o,l,a) \sim \mathcal{D}} \left[\log \pi_{\theta}(\tilde{d} \mid o, l) + \log \pi_{\theta}(\tilde{a} \mid o, l, \tilde{d}) \right], \quad (3)$$

where $\mathcal{D}$ is the robot demonstration dataset and T is the fixed template. Because the sequence is generated autoregressively, each action token causally attends to the preceding language-action prefix, making the prefix a conditioning intermediate rather than an independent auxiliary output. CLAP uses the same VLM backbone, tokenizer, output head, and cross-entropy loss for both language and action tokens, introducing no action expert, auxiliary head, or vocabulary extension.

Action masking augmentation. We consider an optional augmentation that changes the teacher-forced input context without changing the CLAP target sequence. Specifically, we randomly replace a subset of input action tokens with a “?” placeholder while keeping the original tokens as prediction targets. The model must therefore infer the masked values from the image observation, instruction, language-action prefix, and remaining action context. For each sample, the masking fraction is drawn from $\text{Uniform}(0, p_{\max})$, with a 40% probability of applying no masking. Full details are provided in Appendix. We study the effect of masking in Section 5.2; unless otherwise stated, CLAP uses the unmasked configuration.

5 Experiments

5.1 Experimental Setup

Benchmarks. We evaluate CLAP on three benchmarks targeting complementary aspects of VLA performance: (1) in-distribution manipulation, (2) out-of-distribution robustness, and (3) VLM capability analysis. LIBERO [29] evaluates in-distribution manipulation under the standard simulated benchmark protocol; we report results on the four LIBERO suites, covering spatial reasoning, object manipulation, goal-conditioned behavior, and long-horizon tasks. LIBERO-PRO [30] evaluates out-of-distribution robustness by perturbing the original LIBERO tasks, including novel visual object instances, object relocation, instruction rephrasing, and task-configuration changes. VLABench [31] evaluates language-conditioned robot reasoning capabilities; we use it to measure the base Qwen3.5 backbone’s planning and spatial reasoning capabilities before robot fine-tuning, and to analyze how these capabilities relate to downstream VLA performance. Full benchmark details are provided in Appendix.

Table 2: LIBERO benchmark ($h=8$). **Best** result per VLA-0 vs. CLAP pair under the same 1-epoch protocol. Gray rows are prior work under their original training protocols (contextual reference).

Model	Backbone	Size	VLA Pretrain	Spatial $\uparrow$	Object $\uparrow$	Goal $\uparrow$	Long $\uparrow$	Avg $\uparrow$
<i>Ours (1 epoch, identical protocol)</i>								
VLA-0 (repro.)	Qwen3.5	0.8B	–	77.6	87.8	78.4	60.6	76.1
CLAP (Ours)	Qwen3.5	0.8B	–	92.6	97.8	88.4	79.6	89.6 (+13.5)
VLA-0 (repro.)	Qwen3.5	2B	–	77.8	86.6	77.0	62.4	75.9
CLAP (Ours)	Qwen3.5	2B	–	93.0	97.4	90.8	82.0	90.8 (+14.9)
VLA-0 (repro.)	Qwen3.5	4B	–	37.6	84.8	85.8	48.6	64.2
CLAP (Ours)	Qwen3.5	4B	–	88.0	97.4	86.6	67.6	84.9 (+20.7)
<i>Prior work (full training protocol)</i>								
SmolVLA †	SmolVLM2	2.25B	–	93.0	94.0	91.0	77.0	88.8
VLA-0 ‡	Qwen2.5-VL	3B	–	97.0	97.8	96.2	87.6	94.7
$\pi_{0.5}$	PaliGemma	3B	✓	98.8	98.2	98.0	92.4	96.85
OpenVLA	Prismatic	7B	✓	84.7	88.4	79.2	53.7	76.5

† SmolVLA uses community-scale robot pretraining; all sizes in Appendix. ‡ Original VLA-0 uses Qwen2.5-VL-3B and 64 epochs [7].

Comparisons. Our primary baseline is VLA-0, reproduced using the same Qwen3.5 backbones, LIBERO training data, and single-epoch training budget. This matched comparison isolates the effect of the language-action prefix. We also report results from large robot-pretrained VLAs, including OpenVLA [4], π_0 [2], and SmolVLA [5], as contextual reference points. The gray rows in Table 2 indicate methods evaluated under different backbones, training budgets, or robot-pretraining data, and therefore should not be interpreted as matched comparisons.

Implementation. We instantiate CLAP with Qwen3.5 backbones at 0.8B, 2B, and 4B parameters. All models are trained for one epoch on the LIBERO demonstration data across all four suites with full-parameter fine-tuning, using the same action discretization and fine-tuning protocol as VLA-0. Unless otherwise stated, CLAP uses the language-action prefix without additional augmentation. We study random action masking as an optional augmentation in Section 5.2; the unmasked variant is used as the default configuration. Full hyperparameters and evaluation details are provided in Appendix.

5.2 Learning Efficiency Results

Does the language-action prefix improve learning efficiency? Table 2 evaluates performance after one training epoch, isolating the effect of the output representation under matched backbones, data, and training budget. CLAP consistently improves over VLA-0 across all model sizes, with average gains of +13.5, +14.9, and +20.7 points at 0.8B, 2B, and 4B, respectively. At 2B parameters, CLAP achieves the best overall result, reaching 90.8% average success and outperforming VLA-0 by **+14.9 points**. The gains are broad across task suites: 2B CLAP improves Spatial from 77.8% to 93.0%, Object from 86.6% to 97.4%, Goal from 77.0% to 90.8%, and Long from 62.4% to 82.0%. For context, 2B CLAP also exceeds the reported averages of SmolVLA and OpenVLA and approaches stronger robot-pretrained systems such as $\pi_{0.5}$, although these comparisons are not controlled for backbone, data, training budget, or pretraining. These gains come from changing the output representation rather than adding data, model capacity, or architectural components. VLA-0 learns a direct mapping from image-language inputs to bare integer action tokens, which provide precise supervision but little semantic structure. CLAP instead inserts a language-action description before the numeric tokens, giving the model an intermediate target closer to its pretrained generation behavior.

Does action masking provide additional gains? Table 3 ablates whether action masking improves over the language-action prefix alone. Masking is not uniformly beneficial: it reduces average success by 3.9 points at 0.8B and 1.7 points at 2B, but improves the 4B model by 3.2 points, from 84.9% to 88.1%. The drop at smaller scales is mainly due to lower Long-horizon performance, suggesting that 0.8B and 2B models benefit from direct access to the full action-token context. In

Table 3: **Action masking ablation** on LIBERO (1 epoch, $h=8$). We highlight the **best** per size group. ✓ indicates action masking; – indicates no masking. Parentheses in Avg show the change from unmasked CLAP.

Model	Backbone	Size	Masking	Spatial↑	Object↑	Goal↑	Long↑	Avg↑
CLAP	Qwen3.5	0.8B	–	92.6	97.8	88.4	79.6	89.6
			✓	87.6	95.4	90.0	69.8	85.7 (−3.9)
CLAP	Qwen3.5	2B	–	93.0	97.4	90.8	82.0	90.8
			✓	94.8	96.8	91.6	73.2	89.1 (−1.7)
CLAP	Qwen3.5	4B	–	88.0	97.4	86.6	67.6	84.9
			✓	91.6	92.2	89.8	78.8	88.1 (+3.2)

Table 4: LIBERO-PRO OOD generalization across task suites and perturbation types. Results are suite-level average success rates. *Obj*: novel visual instances; *Pos*: object relocation; *Sem*: instruction rephrasing; *Task*: task-configuration change. We highlight the **best** and **second-best** results in each column. All models use Qwen3.5 and are trained for one epoch with $h=8$.

Model	Size	Spatial				Object				Goal				Long				Avg
		Obj	Pos	Sem	Task	Obj	Pos	Sem	Task	Obj	Pos	Sem	Task	Obj	Pos	Sem	Task	
VLA-0 (repro.)	0.8B	70.2	9.4	59.2	50.4	84.4	0.0	85.8	4.6	34.6	0.0	65.6	13.2	27.8	0.0	38.0	9.0	34.5
	2B	67.4	0.6	51.4	44.8	80.6	0.0	74.8	3.4	44.0	1.8	49.4	8.0	31.8	0.0	60.6	8.6	32.9
	4B	39.8	0.0	32.6	31.8	81.0	0.0	86.8	5.4	52.6	0.0	59.2	12.0	34.4	0.0	45.8	8.8	30.6
CLAP w/o masking (Ours)	0.8B	90.0	2.4	68.0	53.8	91.2	0.0	89.0	5.4	44.4	0.2	54.4	13.2	52.0	0.0	64.4	11.2	40.0
	2B	92.6	8.4	66.4	65.8	92.4	0.0	95.8	5.4	57.6	1.8	76.6	14.4	48.2	0.0	68.2	10.2	44.0
	4B	82.4	8.4	81.0	53.2	91.4	0.0	98.8	5.4	38.2	1.0	68.0	17.2	39.8	0.2	64.2	15.4	41.5
CLAP w/ masking (Ours)	0.8B	75.4	0.0	48.4	52.2	93.8	0.0	82.0	5.2	52.6	0.8	56.8	16.4	38.6	0.0	27.2	7.6	34.8
	2B	93.8	6.6	75.4	52.2	91.6	0.0	91.4	3.8	46.0	0.6	71.8	10.4	40.2	0.2	68.4	13.2	41.6
	4B	94.2	10.4	88.2	64.8	87.2	0.0	91.2	4.8	56.6	0.0	79.4	11.4	42.0	0.0	72.2	15.0	44.8

contrast, the 4B model appears better able to exploit the additional reconstruction constraint imposed by masking. These results suggest that action masking should be treated as a validation-dependent augmentation rather than a core component of CLAP.

5.3 Generalization and Capability Transfer Results

Does CLAP’s learning gains transfer out of distribution? Table 4 assesses CLAP’s robustness to out-of-distribution perturbations on LIBERO-Pro. CLAP consistently improves over VLA-0 across model scales, with average OOD gains of +5.5, +11.1, and +10.9 points for unmasked CLAP at 0.8B, 2B, and 4B, respectively. The most striking improvement appears on the Spatial suite with novel visual instances. For 4B models, unmasked CLAP improves over VLA-0 by **+42.6 pt**, while the masked variant improves by **+54.4 pt**. These results suggest that CLAP improves OOD robustness beyond what is captured by standard in-distribution evaluation on LIBERO. Notably, action masking substantially enhances OOD robustness at 4B scale, despite providing only mixed benefits on LIBERO. This suggests that masking can reduce reliance on action-token shortcuts that become brittle under distribution shift.

Does bigger VLM capability mean better transfer? Figure 3 presents VLABench performance of Qwen3.5 backbones prior to robot fine-tuning, evaluated under 1-shot prompting with and without Chain-of-Thought (CoT) [32]. Across most categories, performance generally improves with model size. However, on Complex and Physics Law tasks, the 0.8B and 2B models match or exceed the 4B model under both prompting conditions. This non-monotonic trend persists downstream: CLAP 2B also outperforms CLAP 4B on LIBERO (Table 2). Base VLM capability thus influences VLA performance, yet parameter count alone does not determine transfer quality in the compact regime.

Does reasoning help compact VLAs? Reading the same evaluation along the CoT axis, we find that removing CoT improves performance in most settings; on Physics Law tasks, the 0.8B model even exceeds the larger 2B and 4B models. Output analysis suggests that longer reasoning traces can introduce format violations or loops that diverge from actionable predictions, consistent with

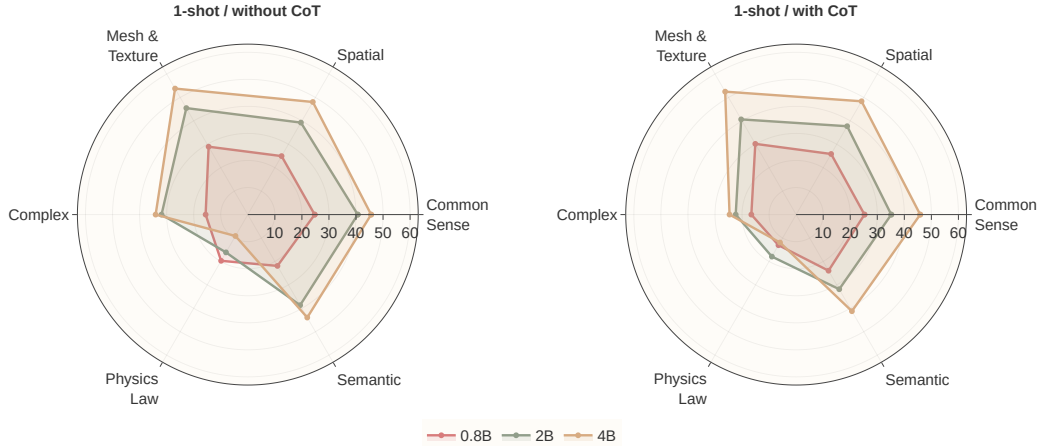


Figure 3: **VLABench capability profiles before robot fine-tuning.** We evaluate Qwen3.5 0.8B, 2B, and 4B under 1-shot prompting, comparing non-CoT prompting on the left with Chain-of-Thought (CoT) prompting on the right. The 2B model shows strong and balanced performance across most capability dimensions, while CoT primarily improves the 4B model on Spatial and Mesh & Texture tasks. These trends motivate CLAP’s structured language-action design for compact VLA fine-tuning.

recent reports that extended reasoning may weaken visual grounding [33]. CLAP avoids relying on free-form reasoning at inference time: its structured language-action template provides a compact intermediate representation before numeric action prediction, keeping smaller variants competitive and the approach attractive for compact, deployment-friendly settings.

6 Conclusion

We presented CLAP, a minimal VLM-to-VLA fine-tuning recipe that addresses output-distribution mismatch by prepending a natural-language action description to each numeric action sequence. With single-epoch training on a single 8-GPU node (under 6.5 hours for all model sizes), 2B CLAP achieves 90.8% on LIBERO, outperforming size-matched VLA-0 by +14.9 points, while also improving OOD robustness on LIBERO-PRO. Multi-scale evaluation across 0.8B, 2B, and 4B reveals that parameter count alone does not determine VLM-to-VLA transfer quality, with the 2B backbone consistently outperforming the 4B variant. We believe that making VLA fine-tuning as lightweight and transparent as standard VLM fine-tuning is a necessary step toward bringing the rapid progress of the VLM community to robot learning, and hope that CLAP provides a useful foundation for this direction. We will release all three model scales from a shared Qwen3.5 lineage to support this direction.

7 Limitations

We limit our study to a single VLM family and limited benchmark suites. Generalization to other backbones, embodiments, and real-world settings remains to be validated. On the inference side, autoregressive token generation is inherently slower than parallel action decoding (e.g. diffusion or flow-matching heads), but deployment techniques in the LLM community, such as speculative decoding and quantization, are directly applicable and remain unexplored in this work. As CLAP preserves the VLM backbone unchanged, VLM-side advances such as distillation, quantization, and new backbone architectures can be incorporated without redesigning the action pipeline.

References

- [1] NVIDIA, :, J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. J. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, J. Jang, Z. Jiang, J. Kautz, K. Kundalia, L. Lao, Z. Li, Z. Lin, K. Lin, G. Liu, E. Llontop, L. Magne, A. Mandlekar, A. Narayan, S. Nasiriany, S. Reed, Y. L. Tan, G. Wang, Z. Wang, J. Wang, Q. Wang, J. Xiang, Y. Xie, Y. Xu, Z. Xu, S. Ye, Z. Yu, A. Zhang, H. Zhang, Y. Zhao, R. Zheng, and Y. Zhu. Gr00t n1: An open foundation model for generalist humanoid robots, 2025. URL <https://arxiv.org/abs/2503.14734>.
- [2] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky. π_0 : A vision-language-action flow model for general robot control, 2024. URL <https://arxiv.org/abs/2410.24164>.
- [3] L. Zha, A. J. Hancock, M. Zhang, T. Yin, Y. Huang, D. Shah, A. Z. Ren, and A. Majumdar. Lap: Language-action pre-training enables zero-shot cross-embodiment transfer, 2026. URL <https://arxiv.org/abs/2602.10556>.
- [4] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. Openvla: An open-source vision-language-action model. In P. Agrawal, O. Kroemer, and W. Burgard, editors, *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 2679–2713. PMLR, 06–09 Nov 2025. URL <https://proceedings.mlr.press/v270/kim25c.html>.
- [5] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadene. Smolvla: A vision-language-action model for affordable and efficient robotics, 2025. URL <https://arxiv.org/abs/2506.01844>.
- [6] J. Wen, Y. Zhu, J. Li, M. Zhu, Z. Tang, K. Wu, Z. Xu, N. Liu, R. Cheng, C. Shen, et al. Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation. *IEEE Robotics and Automation Letters*, 2025.
- [7] A. Goyal, H. Hadfield, X. Yang, V. Blukis, and F. Ramos. Vla-0: Building state-of-the-art vlms with zero modification, 2025. URL <https://arxiv.org/abs/2510.13054>.
- [8] A. J. Hancock, X. Wu, L. Zha, O. Russakovsky, and A. Majumdar. Actions as language: Fine-tuning vlms into vlas without catastrophic forgetting. In *International Conference on Learning Representations*, 2026. URL <https://arxiv.org/abs/2509.22195>.
- [9] Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026. URL <https://qwen.ai/blog?id=qwen3.5>.
- [10] S. Karamcheti, S. Nair, A. Balakrishna, P. Liang, T. Kollar, and D. Sadigh. Prismatic vlms: Investigating the design space of visually-conditioned language models. In *Forty-first International Conference on Machine Learning*, 2024.
- [11] H. Liu, C. Li, Q. Wu, and Y. J. Lee. Visual instruction tuning. *Advances in neural information processing systems*, 36:34892–34916, 2023.
- [12] L. Beyer, A. Steiner, A. S. Pinto, A. Kolesnikov, X. Wang, D. Salz, M. Neumann, I. Al-abdulmohsin, M. Tschannen, E. Bugliarello, T. Unterthiner, D. Keysers, S. Koppula, F. Liu, A. Grycner, A. Gritsenko, N. Houlsby, M. Kumar, K. Rong, J. Eisenschlos, R. Kabra, M. Bauer, M. Bošnjak, X. Chen, M. Minderer, P. Voigtlaender, I. Bica, I. Balazevic, J. Puigcerver, P. Papalampidi, O. Henaff, X. Xiong, R. Soricut, J. Harmsen, and X. Zhai. Paligemma: A versatile 3b vlm for transfer, 2024. URL <https://arxiv.org/abs/2407.07726>.

- [13] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, T. Jackson, S. Jesmonth, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, K.-H. Lee, S. Levine, Y. Lu, U. Malla, D. Manjunath, I. Mordatch, O. Nachum, C. Parada, K. Pertsch, J. Redmond, K. Rao, M. Ryoo, G. Salazar, P. Sanketi, P. Sermanet, J. Singh, A. Singh, R. Soricut, H. Tran, V. Vanhoucke, Q. Vuong, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich. Rt-1: Robotics transformer for real-world control at scale, 2022. URL <https://arxiv.org/abs/2212.06817>.
- [14] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.
- [15] O. M. Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, J. Luo, Y. L. Tan, L. Y. Chen, P. Sanketi, Q. Vuong, T. Xiao, D. Sadigh, C. Finn, and S. Levine. Octo: An open-source generalist robot policy, 2024. URL <https://arxiv.org/abs/2405.12213>.
- [16] K. Pertsch, K. Stachowicz, B. Ichter, D. Driess, S. Nair, Q. Vuong, O. Mees, C. Finn, and S. Levine. FAST: Efficient action tokenization for vision-language-action models. 2025. URL <https://arxiv.org/abs/2501.09747>.
- [17] K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. R. Equi, C. Finn, N. Fusai, M. Y. Galliker, et al. $\pi_0.5$: a vision-language-action model with open-world generalization. In *Conference on Robot Learning*, pages 17–40, 2025.
- [18] S. Stepputtis, J. Campbell, M. Phielipp, S. Lee, C. Baral, and H. Ben Amor. Language-conditioned imitation learning for robot manipulation tasks. *Advances in Neural Information Processing Systems*, 33:13139–13150, 2020.
- [19] A. Brohan, Y. Chebotar, C. Finn, K. Hausman, A. Herzog, D. Ho, J. Ibarz, A. Irpan, E. Jang, R. Julian, et al. Do as i can, not as i say: Grounding language in robotic affordances. In *Conference on robot learning*, pages 287–318. PMLR, 2023.
- [20] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Thompson, I. Mordatch, Y. Chebotar, et al. Inner monologue: Embodied reasoning through planning with language models. In *Conference on Robot Learning*, pages 1769–1782. PMLR, 2023.
- [21] D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Thompson, Q. Vuong, T. Yu, W. Huang, Y. Chebotar, P. Sermanet, D. Duckworth, S. Levine, V. Vanhoucke, K. Hausman, M. Toussaint, K. Greff, A. Zeng, I. Mordatch, and P. Florence. Palm-e: An embodied multimodal language model, 2023. URL <https://arxiv.org/abs/2303.03378>.
- [22] M. Zawalski, W. Chen, K. Pertsch, O. Mees, C. Finn, and S. Levine. Robotic control via embodied chain-of-thought reasoning. In *Conference on Robot Learning*, pages 3157–3181. PMLR, 2025.
- [23] W. Chen, S. Belkhale, S. Mirchandani, K. Pertsch, D. Driess, O. Mees, and S. Levine. Training strategies for efficient embodied reasoning. In *Conference on Robot Learning*, pages 365–391. PMLR, 2025.
- [24] R. Zheng, Y. Liang, S. Huang, J. Gao, H. Daumé III, A. Kolobov, F. Huang, and J. Yang. Tracevla: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies. In *International Conference on Learning Representations*, volume 2025, pages 54277–54296, 2025.
- [25] D. Driess, J. T. Springenberg, brian ichter, L. YU, A. Li-Bell, K. Pertsch, A. Z. Ren, H. Walke, Q. Vuong, L. X. Shi, and S. Levine. Knowledge insulating vision-language-action models: Train fast, run fast, generalize better. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=cb0xbZ3APM>.

- [26] S. Grover, A. Gopalkrishnan, B. Ai, H. I. Christensen, H. Su, and X. Li. Enhancing generalization in vision-language-action models by preserving pretrained representations, 2025. URL <https://arxiv.org/abs/2509.11417>.
- [27] M. Reuss, H. Zhou, M. Rühle, O. E. Yağmurlu, F. Otto, and R. Lioutikov. Flower: Democratizing generalist robot policies with efficient vision-language-flow models. In J. Lim, S. Song, and H.-W. Park, editors, *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 3736–3761. PMLR, 27–30 Sep 2025. URL <https://proceedings.mlr.press/v305/reuss25a.html>.
- [28] Y. Wang, P. Ding, L. Li, C. Cui, Z. Ge, X. Tong, W. Song, H. Zhao, W. Zhao, P. Hou, et al. Vla-adapter: An effective paradigm for tiny-scale vision-language-action model. In *Proceedings of the AAAI conference on artificial intelligence*, volume 40, pages 18638–18646, 2026.
- [29] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.
- [30] X. Zhou, Y. Xu, G. Tie, Y. Chen, G. Zhang, D. Chu, P. Zhou, and L. Sun. Libero-pro: Towards robust and fair evaluation of vision-language-action models beyond memorization, 2026. URL <https://arxiv.org/abs/2510.03827>.
- [31] S. Zhang, Z. Xu, P. Liu, X. Yu, Y. Li, Q. Gao, Z. Fei, Z. Yin, Z. Wu, Y.-G. Jiang, and X. Qiu. Vlabench: A large-scale benchmark for language-conditioned robotics manipulation with long-horizon reasoning tasks. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 11142–11152, October 2025.
- [32] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS ’22*, Red Hook, NY, USA, 2022. Curran Associates Inc. ISBN 9781713871088.
- [33] S. Raghu and S. Pandey. Don’t blink: Evidence collapse during multimodal reasoning, 2026. URL <https://arxiv.org/abs/2604.04207>.

article

corl₂026

myconfig pifont amsmath

A Multimedia Material

Please see the video with audio alongside with this PDF. We enclose multimedia material consisting of a graphical abstract summarizing the CLAP framework and real-robot (UR5e) rollouts.

B Real-Robot Deployment

The main experiments evaluate CLAP on simulated benchmarks (LIBERO [29], LIBERO-PRO [30]). Here we provide a preliminary assessment of whether our direct VLM-to-VLA recipe transfers to a physical robot. We deploy CLAP on a UR5e manipulator for cluttered pick-and-place, fine-tuning on a limited real-robot demonstration set to examine how model scale affects task performance in real-world scenarios.

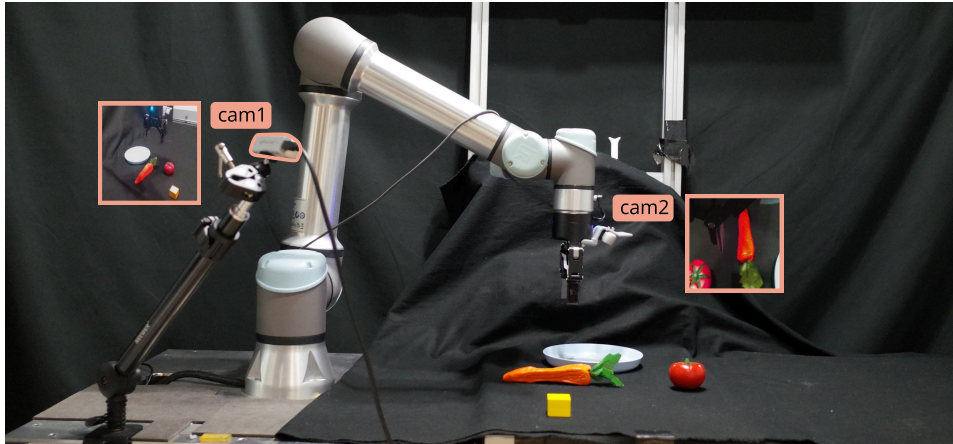


Figure 4: **Real-Robot Setup.** A UR5e arm with a Robotiq 2F-85 gripper performs pick-and-place tasks in cluttered tabletop scenes. Cam1 (fixed side view) and cam2 (wrist-mounted) provide the two RGB observations used as model input. Insets show example frames from each camera.

B.1 Real-Robot Setup

Figure 4 shows a real-robot setup to validate our model. We use a UR5e collaborative robot arm equipped with a Robotiq 2F-85 parallel gripper. A wrist-mounted RealSense D435 provides an eye-in-hand view, and a second RealSense D435 provides a fixed third-person view. CLAP runs on a workstation with an NVIDIA GeForce RTX 4090 GPU, served via FastAPI, and queried by a ROS client at inference time. The ROS client runs a synchronous observe-query-act loop, at each query, the current side- and wrist-camera images and the language instruction are sent to the inference server via an HTTP POST request, which returns an eight-step action chunk. Each action encodes a Cartesian end-effector delta (position, axis-angle orientation, and gripper state). The client executes each eight-step action chunk in open loop before re-querying the VLA. At the controller’s 15 Hz rate, this yields an effective query rate of ~ 1.9 Hz.

B.2 Data Collection.

During data collection, we collect demonstrations on the UR5e via VR controller-based teleoperation. The operator controls the robot using an HTC Vive controller, whose motion is mapped to

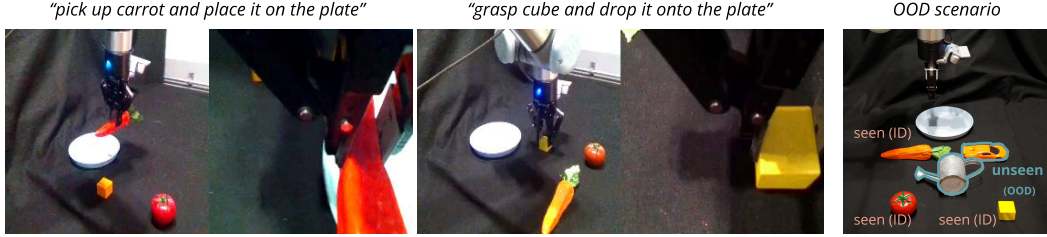


Figure 5: **Pick-and-Place Task in Cluttered Scene.** Left three: pick-and-place tasks in cluttered scenes with randomized object layouts. Right: an out-of-distribution scenario with unseen objects.

Cartesian end-effector delta commands at 50 Hz. Data is recorded from two RGB camera streams, consisting of a fixed external camera and a wrist-mounted camera, together with the end-effector pose, gripper state, and a natural-language instruction at each timestep. Each episode lasts up to 1500 steps. For each episode, the scene layout is generated from a seeded episode plan. All three objects (a carrot, a tomato, and a cube) are present simultaneously and are placed at three of five discrete table positions. We use 50 distinct object-to-position assignments from this layout space. Before each demonstration, the operator arranges the scene according to the pre-generated layout. We collect 120 demonstrations in total, with 40 episodes for each target object. The instruction template is randomized per episode from four templates: “place {target} on the plate,” “put {target} onto the plate,” “pick up {target} and place it on the plate,” and “grasp {target} and drop it onto the plate.”

B.3 Training Protocol

We fine-tune Qwen3.5-0.8B and 2B directly from their pretrained VLM checkpoints using the CLAP framework on the 120 real-robot demonstrations. Notably, we do not initialize from the LIBERO-finetuned models used in the main experiments: because the base VLMs are pretrained on real images, we focus on assessing how much performance can be extracted from VLM-to-VLA adaptation using only real-world data, without an intermediate simulation fine-tuning stage.

B.4 Task Design and Evaluation

We evaluate pick-and-place tasks with three training objects (carrot, cube, tomato) under multiple instruction templates. Table 5 summarizes all objects used in the evaluation. Each model is tested under two conditions:

- **In-domain (ID):** Object sets and layouts seen during training.
- **Out-of-domain (OOD):** Unseen distractor objects (a silver watering can, an orange toy car) are introduced into the scene.

Table 5: Objects used in real-robot evaluation. Seen objects appear in the training demonstrations as pick targets. Unseen objects are introduced as distractors only and are never picked.

Category	Object	Description
Seen (pick target)	Carrot	Orange toy carrot; elongated shape
	Cube	Plain yellow wooden cube; uniform color and geometry
	Tomato	Red toy tomato; spherical shape
Unseen (distractor)	Silver watering can	Geometrically complex (spout, handle), dissimilar to any training object
	Orange toy car	Orange color visually confusable with the carrot, testing robustness to appearance-similar distractors

Table 6 reports success rates over 20 trials per condition. The 2B model achieves the highest success rate (60%) followed by 0.8B (35%), consistent with simulation results where the 2B backbone also

outperforms the 0.8B model. The overall success rates are substantially lower than in simulation, reflecting the additional complexity of real-world perception and control with only 120 training demonstrations. Under OOD conditions with unseen distractor objects, the 2B model maintains its in-domain success rate (60%), while the 0.8B model drops substantially to 10%, suggesting that the larger backbone is more robust to visual distractors. This capacity gap is substantially wider than observed on LIBERO (0.8B: 89.6%, 2B: 90.8%), suggesting that simulation benchmarks may underestimate the benefit of model capacity, and that real-world task complexity may correlate more strongly with backbone size.

Table 6: Real-robot pick-and-place success rates (%). 20 trials per condition.

Model	Backbone	Size	ID (%)	OOD (%)
CLAP	Qwen3.5	0.8B	35.0	10.0
CLAP	Qwen3.5	2B	60.0	60.0

The results suggest two complementary directions for future work. The first is to further explore the minimal-recipe regime: improving real-world performance with limited demonstrations through better data augmentation, input robustness, or more effective use of the pretrained VLM’s capabilities, without introducing additional architectural components. The second is to scale up robot data while retaining the language-action representation [33], and investigating whether CLAP’s zero-modification architecture benefits similarly from larger data remains an open question.

C Inference Latency

Table 7 reports end-to-end inference latency per policy query, measured as the mean over ten consecutive queries after one warm-up query on an RTX 4090. The 0.8B and 2B models exhibit near-identical latency, indicating that inference time is dominated by token generation overhead rather than model size at these scales. The 4B model is moderately slower (~ 6.0 s). With $h=8$ (open-loop execution of the full action chunk), effective control rates range from 1.3 to 1.9 Hz.

Comparing CLAP 0.8B against VLA-0 0.8B on the same backbone isolates the overhead of the language-action prefix: CLAP adds approximately 1.0 s per query (+32%), corresponding to the additional ~ 149 tokens generated for the prefix. GPU memory consumption is identical, confirming that the prefix introduces no architectural overhead. As CLAP preserves the VLM backbone unmodified, standard LLM acceleration techniques such as speculative decoding and quantization are directly applicable and could substantially reduce this overhead.

Table 7: Inference latency per policy query (action horizon $h = 8$, RTX 4090).

Model	Backbone	Size	Latency (s)	Eff. Hz ($k=8$)	Peak GPU Memory (GiB)
VLA-0 [7]	Qwen3.5	0.8B	3.205	2.50	3.9
CLAP	Qwen3.5	0.8B	4.233	1.89	3.9
CLAP	Qwen3.5	2B	4.307	1.86	9.1
CLAP	Qwen3.5	4B	6.013	1.33	18.0

D Training Details

D.1 Training Hyperparameters

In Section 5 of the main manuscript, we fine-tune Qwen3.5-0.8,2,4B VLM with full-parameter updates on $8 \times$ NVIDIA H100 GPUs. Each GPU processes a micro-batch of 16, yielding a global effective batch size of 128 per optimizer step. We define **one epoch** as a single pass over the LIBERO-spatial, object, goal, long training set under our augmentation pipeline, which in our configuration corresponds to 17,000 gradient steps (about 2.18M training samples).

Table 8: Training hyperparameters shared across all CLAP and VLA-0 experiments.

Hyperparameter	Value
Optimizer	AdamW
Training steps	17,000
Learning rate	5.0×10^{-6} (constant schedule)
Weight decay	10^{-10}
Effective batch size	128 (16 per GPU $\times$ 8 GPUs, DDP)
Mixed precision	bfloat16
Attention	Flash Attention 2
Image augmentation	Random crop (scale 0.875), color jitter
Action masking rate	0.4

D.2 Loss Convergence vs. Task Saturation

Section 5.2 of the main text reports LIBERO performance after a single training epoch (approximately four hours on 8 $\times$ H200 GPUs). In extended runs up to six epochs, fig. 6 demonstrates that the closed-loop success saturates at roughly 90% after the first epoch; subsequent epochs continue to reduce training loss but do not improve task success, suggesting overfitting to the demonstration distribution. We therefore adopt the single-epoch checkpoint throughout.

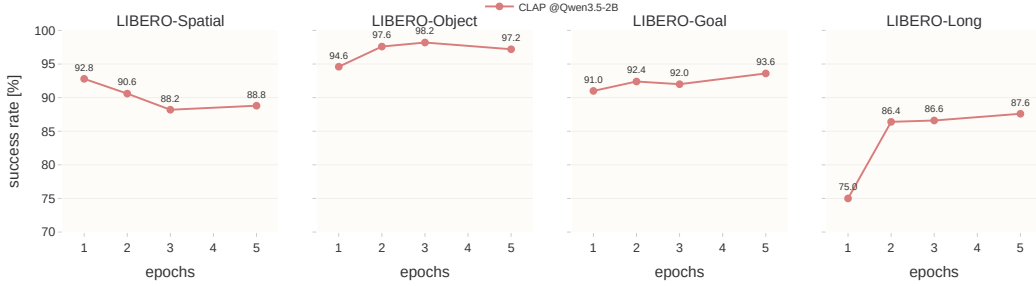


Figure 6: **LIBERO Success Rate** of CLAP@Qwen3.5-2B across training epochs on the four LIBERO suites. Success exceeds 90% on Spatial, Object, and Goal already at epoch 1, while Long benefits from one additional epoch before saturating.

E Evaluation Protocol

Each observation encodes two camera views (agentview and wrist) tiled into a single 224 $\times$ 224 image. The model produces the full `<think>...</think>` `action_tokens` sequence in one autoregressive pass using Qwen3.5’s built-in extended-thinking mode (`enable_thinking=True`); only the 56 integer tokens after the closing `</think>` tag are forwarded to the robot controller. All eight predicted action steps execute open-loop before the model is re-queried (action horizon $h=8$).

F CLAP Language Target Construction

During training, the language-action prefix is generated deterministically from each ground-truth action chunk using a fixed template-based method [3] that summarizes steps of numerical actions into a short natural-language summary.

Inputs. Let $\mathbf{a} \in \mathbb{R}^{7 \times h}$ denote an action chunk of horizon h , where each $a_t = (\delta x_t, \delta y_t, \delta z_t, \delta \phi_t, \delta \theta_t, \delta \psi_t, g_t)$ contains end-effector translational deltas in meters, rotational deltas in radians (roll, pitch, yaw), and a gripper state $g_t \in \{0, 1\}$.

486 **Aggregation.** We sum the per-step deltas across the chunk and convert to human-readable units:

$$\Delta x = \text{round} \left(100 \sum_{t=1}^h \delta x_t \right) [\text{cm}], \quad \text{and analogously for } \Delta y, \Delta z, \quad (4)$$

$$\Delta \phi = \text{round}_{10} \left(\frac{180}{\pi} \sum_{t=1}^h \delta \phi_t \right) [\text{deg}], \quad \text{and analogously for } \Delta \theta, \Delta \psi, \quad (5)$$

487 where round_{10} rounds to the nearest multiple of 10° . The gripper bit is taken from the last step,
488 $g_H \geq 0.5 \Rightarrow \text{open}$, otherwise close.

489 **Phrasing.** Each non-zero aggregated component is mapped to a phrase whose direction word is
490 selected by the sign of the underlying delta:

Axis	Sign > 0	Sign < 0
Δx	move forward $ \Delta x $ cm	move back $ \Delta x $ cm
Δy	move left $ \Delta y $ cm	move right $ \Delta y $ cm
491 Δz	move up $ \Delta z $ cm	move down $ \Delta z $ cm
$\Delta \phi$	tilt left $ \Delta \phi $ degrees	tilt right $ \Delta \phi $ degrees
$\Delta \theta$	tilt back $ \Delta \theta $ degrees	tilt forward $ \Delta \theta $ degrees
$\Delta \psi$	rotate counterclockwise $ \Delta \psi $ degrees	rotate clockwise $ \Delta \psi $ degrees

492 The selected phrases, followed by the gripper phrase, are joined with " , " into a single sentence d .

493 **Final target.** The CLAP target is the language summary wrapped in `<think>...</think>` tags
494 and followed by the same discretised numeric action tokens used by VLA-0:

$$\text{<think>} d \text{</think>} n_1 n_2 \dots n_{7 \cdot h}$$

495 where n_i are the per-dimension action bin indices. Concretely, for $h = 8$ with a small forward and
496 downward motion and the gripper closing, the target looks like

497 `<think>move forward 3 cm, move down 2 cm, close gripper</think> 17 13 ... 0`

498 At inference, the prefix up to and including the last `</think>` is stripped and the remaining numeric
499 tokens are decoded back to a continuous action chunk.

500 G Action Masking Details

501 Following prior work [7], each action chunk is discretized into 1000 bins per dimension and se-
502 rialized as space-separated integers. Cross-entropy loss is computed only over the assistant turn;
503 all prompt-side tokens (system message, images, instruction, and padding) are masked out. As an
504 optional augmentation, a random fraction of input action tokens are replaced with a ? placeholder
505 while keeping the original values as prediction targets, forcing the model to reconstruct masked
506 actions from visual and linguistic context. For each sample, the masking fraction is drawn from
507 $\text{Uniform}(0, p_{\max})$ with $p_{\max}=0.4$. For CLAP, this augmentation is restricted to the numeric tail
508 following `</think>` to avoid misalignment within the language-action prefix. At inference, a
509 logits processor constrains generation to digits, spaces, and `eos`; in CLAP mode this constraint is
510 suspended during the `<think>` span and re-enabled for the numeric tail.

511 H Benchmark Details

512 **LIBERO.** LIBERO [29] is a simulated manipulation benchmark comprising four task suites: Spa-
513 tial, Object, Goal, and Long. Each suite contains 10 tasks with 50 demonstrations per task. We
514 follow the standard evaluation protocol and report success rates averaged over 20 rollouts per task.



Figure 7: **VLABench capability profiles under 0-shot prompting.** Same evaluation as Figure 3 in the main paper (1-shot) but without an in-context example. The overall capability ranking across model scales is consistent with the 1-shot setting, with the 4B model showing balanced performance across most dimensions. However, Physical Law accuracy, which remains modest but nonzero under 1-shot prompting (Figure 3 in the main paper), drops to near zero for all three scales without the in-context example, suggesting that even this limited performance relies on the provided demonstration.

515 **LIBERO-PRO.** LIBERO-PRO [30] extends LIBERO with four perturbation types designed to
516 stress-test out-of-distribution robustness. *Obj* replaces objects with novel visual instances of the
517 same category. *Pos* relocates objects to positions unseen during training. *Sem* rephrases task in-
518 structions while preserving semantics. *Task* modifies the task configuration itself (e.g., goal location
519 or object arrangement), representing the hardest perturbation type. All four perturbation types are
520 applied to each of the four LIBERO suites.

521 **VLABench.** VLABench [31] evaluates task planning across 45 tasks covering six capability di-
522 mensions: Common Sense & World Knowledge (background knowledge about object properties and
523 everyday scenarios), Spatial Understanding (precise spatial reasoning about object positions and
524 orientations), Mesh & Texture Understanding (distinguishing objects by shape or surface appear-
525 ance), Semantic Understanding (language-grounded category-level reasoning), Physics Law (tasks
526 governed by physics constraints such as gravity, support, and rigidity), and Complex (multi-step in-
527 ference combining multiple dimensions). Each dimension covers 100 episodes; results are averaged
528 over all episodes within each dimension.

529 The main paper reports 1-shot evaluation of the Qwen3.5 backbones before robot fine-tuning. We
530 repeat the evaluation under 0-shot prompting (Figure 7) and find that the overall capability ranking
531 is consistent, although Physical Law accuracy drops to near zero without the in-context example.

532 I SmolVLA Full Size Comparison

533 Table 9 reports all three SmolVLA sizes alongside CLAP for reference. SmolVLA uses community-
534 scale robot pretraining, whereas CLAP uses no robot pretraining beyond the LIBERO demonstra-
535 tions. Despite this difference, CLAP 0.8B and 2B match or exceed all three SmolVLA scales on
536 average success rate with single-epoch training, while CLAP 4B falls below SmolVLA 2.25B, con-
537 sistent with the non-monotonic scaling observed in the main paper.

Table 9: SmolVLA all sizes vs. CLAP on LIBERO (1 epoch, $h=8$). Gray rows show SmolVLA under its full training protocol (contextual reference only).

Model	Backbone	Size	VLA Pretrain	Spatial $\uparrow$	Object $\uparrow$	Goal $\uparrow$	Long $\uparrow$	Avg $\uparrow$
<i>Ours (1 epoch, identical protocol)</i>								
CLAP	Qwen3.5	0.8B	–	92.6	97.8	88.4	79.6	89.6
CLAP	Qwen3.5	2B	–	93.0	97.4	90.8	82.0	90.8
CLAP	Qwen3.5	4B	–	88.0	97.4	86.6	67.6	84.9
<i>SmolVLA (full training protocol)</i>								
SmolVLA $\dagger$	SmolVLM2	0.24B	–	87.0	93.0	88.0	63.0	82.8
SmolVLA $\dagger$	SmolVLM2	0.45B	–	90.0	96.0	92.0	71.0	87.3
SmolVLA $\dagger$	SmolVLM2	2.25B	–	93.0	94.0	91.0	77.0	88.8

$\dagger$ SmolVLA uses community-scale robot pretraining.

References

- [1] NVIDIA, :, J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. J. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, J. Jang, Z. Jiang, J. Kautz, K. Kundalia, L. Lao, Z. Li, Z. Lin, K. Lin, G. Liu, E. Llontop, L. Magne, A. Mandlekar, A. Narayan, S. Nasiriany, S. Reed, Y. L. Tan, G. Wang, Z. Wang, J. Wang, Q. Wang, J. Xiang, Y. Xie, Y. Xu, Z. Xu, S. Ye, Z. Yu, A. Zhang, H. Zhang, Y. Zhao, R. Zheng, and Y. Zhu. Gr00t n1: An open foundation model for generalist humanoid robots, 2025. URL <https://arxiv.org/abs/2503.14734>.
- [2] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky. π_0 : A vision-language-action flow model for general robot control, 2024. URL <https://arxiv.org/abs/2410.24164>.
- [3] L. Zha, A. J. Hancock, M. Zhang, T. Yin, Y. Huang, D. Shah, A. Z. Ren, and A. Majumdar. Lap: Language-action pre-training enables zero-shot cross-embodiment transfer, 2026. URL <https://arxiv.org/abs/2602.10556>.
- [4] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. Openvla: An open-source vision-language-action model. In P. Agrawal, O. Kroemer, and W. Burgard, editors, *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 2679–2713. PMLR, 06–09 Nov 2025. URL <https://proceedings.mlr.press/v270/kim25c.html>.
- [5] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadene. Smolvla: A vision-language-action model for affordable and efficient robotics, 2025. URL <https://arxiv.org/abs/2506.01844>.
- [6] J. Wen, Y. Zhu, J. Li, M. Zhu, Z. Tang, K. Wu, Z. Xu, N. Liu, R. Cheng, C. Shen, et al. Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation. *IEEE Robotics and Automation Letters*, 2025.
- [7] A. Goyal, H. Hadfield, X. Yang, V. Blukis, and F. Ramos. Vla-0: Building state-of-the-art vlars with zero modification, 2025. URL <https://arxiv.org/abs/2510.13054>.
- [8] A. J. Hancock, X. Wu, L. Zha, O. Russakovsky, and A. Majumdar. Actions as language: Fine-tuning vlms into vlars without catastrophic forgetting. In *International Conference on Learning Representations*, 2026. URL <https://arxiv.org/abs/2509.22195>.
- [9] Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026. URL <https://qwen.ai/blog?id=qwen3.5>.

- [10] S. Karamcheti, S. Nair, A. Balakrishna, P. Liang, T. Kollar, and D. Sadigh. Prismatic vlms: Investigating the design space of visually-conditioned language models. In *Forty-first International Conference on Machine Learning*, 2024.
- [11] H. Liu, C. Li, Q. Wu, and Y. J. Lee. Visual instruction tuning. *Advances in neural information processing systems*, 36:34892–34916, 2023.
- [12] L. Beyer, A. Steiner, A. S. Pinto, A. Kolesnikov, X. Wang, D. Salz, M. Neumann, I. Al-abdulmohsin, M. Tschannen, E. Bugliarello, T. Unterthiner, D. Keysers, S. Koppula, F. Liu, A. Grycner, A. Gritsenko, N. Houlsby, M. Kumar, K. Rong, J. Eisenschlos, R. Kabra, M. Bauer, M. Bošnjak, X. Chen, M. Minderer, P. Voigtlaender, I. Bica, I. Balazevic, J. Puigcerver, P. Papalampidi, O. Henaff, X. Xiong, R. Soricut, J. Harmsen, and X. Zhai. Paligemma: A versatile 3b vlm for transfer, 2024. URL <https://arxiv.org/abs/2407.07726>.
- [13] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, T. Jackson, S. Jesmonth, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, K.-H. Lee, S. Levine, Y. Lu, U. Malla, D. Manjunath, I. Mordatch, O. Nachum, C. Parada, K. Pertsch, J. Redmond, K. Rao, M. Ryoo, G. Salazar, P. Sanketi, P. Sermanet, J. Singh, A. Singh, R. Soricut, H. Tran, V. Vanhoucke, Q. Vuong, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich. Rt-1: Robotics transformer for real-world control at scale, 2022. URL <https://arxiv.org/abs/2212.06817>.
- [14] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.
- [15] O. M. Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, J. Luo, Y. L. Tan, L. Y. Chen, P. Sanketi, Q. Vuong, T. Xiao, D. Sadigh, C. Finn, and S. Levine. Octo: An open-source generalist robot policy, 2024. URL <https://arxiv.org/abs/2405.12213>.
- [16] K. Pertsch, K. Stachowicz, B. Ichter, D. Driess, S. Nair, Q. Vuong, O. Mees, C. Finn, and S. Levine. FAST: Efficient action tokenization for vision-language-action models. 2025. URL <https://arxiv.org/abs/2501.09747>.
- [17] K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. R. Equi, C. Finn, N. Fusai, M. Y. Galliker, et al. $\pi_0.5$: a vision-language-action model with open-world generalization. In *Conference on Robot Learning*, pages 17–40, 2025.
- [18] S. Stepputtis, J. Campbell, M. Phielipp, S. Lee, C. Baral, and H. Ben Amor. Language-conditioned imitation learning for robot manipulation tasks. *Advances in Neural Information Processing Systems*, 33:13139–13150, 2020.
- [19] A. Brohan, Y. Chebotar, C. Finn, K. Hausman, A. Herzog, D. Ho, J. Ibarz, A. Irpan, E. Jang, R. Julian, et al. Do as i can, not as i say: Grounding language in robotic affordances. In *Conference on robot learning*, pages 287–318. PMLR, 2023.
- [20] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Thompson, I. Mordatch, Y. Chebotar, et al. Inner monologue: Embodied reasoning through planning with language models. In *Conference on Robot Learning*, pages 1769–1782. PMLR, 2023.
- [21] D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Thompson, Q. Vuong, T. Yu, W. Huang, Y. Chebotar, P. Sermanet, D. Duckworth, S. Levine, V. Vanhoucke, K. Hausman, M. Toussaint, K. Greff, A. Zeng, I. Mordatch, and P. Florence. Palm-e: An embodied multimodal language model, 2023. URL <https://arxiv.org/abs/2303.03378>.
- [22] M. Zawalski, W. Chen, K. Pertsch, O. Mees, C. Finn, and S. Levine. Robotic control via embodied chain-of-thought reasoning. In *Conference on Robot Learning*, pages 3157–3181. PMLR, 2025.

- [23] W. Chen, S. Belkhale, S. Mirchandani, K. Pertsch, D. Driess, O. Mees, and S. Levine. Training strategies for efficient embodied reasoning. In *Conference on Robot Learning*, pages 365–391. PMLR, 2025.
- [24] R. Zheng, Y. Liang, S. Huang, J. Gao, H. Daumé III, A. Kolobov, F. Huang, and J. Yang. Tracevla: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies. In *International Conference on Learning Representations*, volume 2025, pages 54277–54296, 2025.
- [25] D. Driess, J. T. Springenberg, brian ichter, L. YU, A. Li-Bell, K. Pertsch, A. Z. Ren, H. Walke, Q. Vuong, L. X. Shi, and S. Levine. Knowledge insulating vision-language-action models: Train fast, run fast, generalize better. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=cb0xbZ3APM>.
- [26] S. Grover, A. Gopalkrishnan, B. Ai, H. I. Christensen, H. Su, and X. Li. Enhancing generalization in vision-language-action models by preserving pretrained representations, 2025. URL <https://arxiv.org/abs/2509.11417>.
- [27] M. Reuss, H. Zhou, M. Rühle, O. E. Yağmurlu, F. Otto, and R. Lioutikov. Flower: Democratizing generalist robot policies with efficient vision-language-flow models. In J. Lim, S. Song, and H.-W. Park, editors, *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 3736–3761. PMLR, 27–30 Sep 2025. URL <https://proceedings.mlr.press/v305/reuss25a.html>.
- [28] Y. Wang, P. Ding, L. Li, C. Cui, Z. Ge, X. Tong, W. Song, H. Zhao, W. Zhao, P. Hou, et al. Vla-adapt: An effective paradigm for tiny-scale vision-language-action model. In *Proceedings of the AAAI conference on artificial intelligence*, volume 40, pages 18638–18646, 2026.
- [29] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.
- [30] X. Zhou, Y. Xu, G. Tie, Y. Chen, G. Zhang, D. Chu, P. Zhou, and L. Sun. Libero-pro: Towards robust and fair evaluation of vision-language-action models beyond memorization, 2026. URL <https://arxiv.org/abs/2510.03827>.
- [31] S. Zhang, Z. Xu, P. Liu, X. Yu, Y. Li, Q. Gao, Z. Fei, Z. Yin, Z. Wu, Y.-G. Jiang, and X. Qiu. Vlabench: A large-scale benchmark for language-conditioned robotics manipulation with long-horizon reasoning tasks. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 11142–11152, October 2025.
- [32] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS ’22*, Red Hook, NY, USA, 2022. Curran Associates Inc. ISBN 9781713871088.
- [33] S. Raghu and S. Pandey. Don’t blink: Evidence collapse during multimodal reasoning, 2026. URL <https://arxiv.org/abs/2604.04207>.